Implementation_2 Final_Report

An Intelligent Image Classification System for Food Recognition

Team – Aivis

Team Member	Registration No	Index No
S.Keerththika	ICT/2022/007	5615
M.F.F. Fazeena	ICT/2022/024	5631
M.S.S.J.Banu	ICT/2022/063	5668
V.Yathursana	ICT/2022/112	5714
R.Anusha	ICT/2022/134	5735

Department of Computing

Faculty of Applied Sciences

Rajarata University of Sri Lanka

Table of Contents

1. Introduction **5**
2. Dataset Description **5**
3. Model Architectures and Experiments **7**
3.1. Experiment 1 - Baseline CNN Model **7**
3.2. Experiment 2 - Improved Model **9**
4. Training Process **12**
5. Experimental Results **14**
5.1. Results of Implementation_1 ..... **14**
5.2. Results of Implementation_2 ..... **15**
5.3. Class Wise Result Comparison **15**
6. Overfitting Analysis **19**
7. Final Model and Results **20**
8. Discussion **22**
9. Conclusion and Future Work **23**
10. GitHub Link **23**

List of Figures

Figure 1:Dataset split confirmation 6
Figure 2:Classes printed in console 6
Figure 3:Sample training images 6
Figure 4:Class distribution table and bar chart 7
Figure 5: Normalization output 8
Figure 6:Baseline CNN model summary 9
Figure 7:Classes and number of classes in Implementation 2 11
Figure 8:Sample training images for Implementation 2 11
Figure 9:Class weights output 11
Figure 10:Training log of Implementation 2 with learning rate reduction and best epoch restoration 13
Figure 11:Accuracy and loss curves of Implementation 1 13
Figure 12:Accuracy and loss curves of Implementation 2 14
Figure 13:Test accuracy and loss output of Implementation 1 15
Figure 14:Classification report of Implementation 1 16
Figure 15:Confusion matrix of Implementation 1 16
Figure 16:Sample predictions of Implementation 1 17
Figure 17:Test accuracy and loss output of Implementation 2 17
Figure 18:Classification report of Implementation 2 18
Figure 19:Confusion matrix of Implementation 2 18
Figure 20:Sample predictions of Implementation 2 19
Figure 21:Improved model summary 21
Figure 22:Final model saving confirmation 21

List of Tables

Table 1:Dataset Summary 5
Table 2:Baseline Model Summary 8
Table 3:Improved Model Summary 10
Table 4:Training Configuration Comparison 12
Table 5:Implementation 1 Test Results 14
Table 6:Implementation 2 Test Results 15
Table 7:Final Selected Model Details 20

1. Introduction

This project was carried out to develop an intelligent image classification system that can recognize food images correctly. The system was designed to classify images into five predefined categories: Chapathi, Dhal, Idli, Jelebi, and Samosa. The work was completed in two stages. First, a baseline model was developed using a simple Convolutional Neural Network (CNN). Then, the model was improved in Implementation 2 using a more advanced architecture and optimization techniques.

The main purpose of this project was not only to obtain a high accuracy score, but also to understand the full learning process of image classification model development. Therefore, this report documents the complete project evolution, including the baseline model, the improved model, the training process, the results of both implementations, the problems observed, and the methods used to improve performance. This approach follows the project guideline, which emphasizes documenting important experiments, model versions, and the reasoning behind the improvements rather than showing only the final result.

2. Dataset Description

The dataset used in this project consists of five food classes. They are Chapathi, Dhal, Idli, Jelebi, and Samosa. The images were organized into separate folders according to their class labels. This folder based structure allowed TensorFlow to automatically assign the correct labels while loading the dataset.

To evaluate the model fairly, the dataset was divided into training, validation, and test sets using the `splitfolders` library. The split ratio used for both implementations was 70% for training, 15% for validation, and 15% for testing. The dataset structure and class distribution were checked before model training. The class counts show that the dataset is fairly balanced, although Samosa has fewer images than the other classes. This imbalance became more important in the second implementation and was handled using class weights.

Table 1:Dataset Summary

Class	Training	Validation	Testing	Total
Chapathi	546	117	118	781
Dhal	513	109	111	733
Idli	489	105	106	700
Jelebi	440	94	95	629
Samosa	341	73	74	488

The dataset preparation stage confirmed that all five classes were correctly loaded and that the data pipeline was ready for training. Sample images were also visualized to verify the quality and diversity of the dataset. The Implementation 1 report already documented this dataset preparation clearly, including the split confirmation, class names, sample image grid, and class distribution chart.

```
Dataset split saved to: C:\Users\LENOVO\Desktop\IIS_Project\Splited_DataSet
```

Figure 1:Dataset split confirmation

```
Classes: ['Chapathi', 'Dhal', 'Idli', 'Jelebi', 'Samosa']
```

Figure 2:Classes printed in console

Figure 3:Sample training images

```
Class Distribution Table:
Class           Train   Val     Test    Total
Chapathi                546     117     118     781
Dhal            513     109     111     733
Idli            489     105     106     700
Jelebi          440     94      95      629
Samosa          341     73      74      488
```

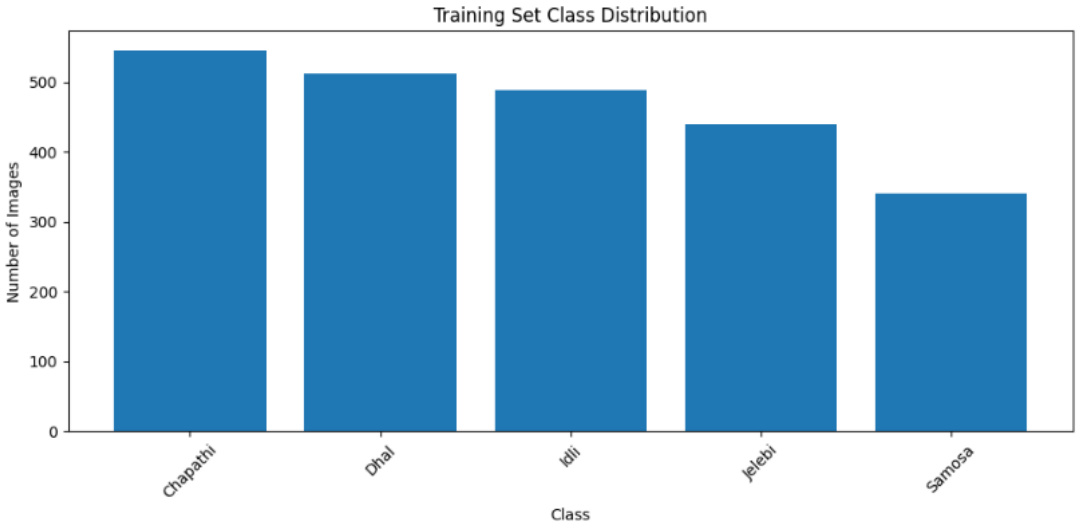

Figure 4:Class distribution table and bar chart

3. Model Architectures and Experiments

This project involved two major experiments. The first experiment used a simple CNN as the baseline model. The second experiment introduced a stronger model based on MobileNetV2 transfer learning together with several optimization techniques. The purpose of these experiments was to observe how the system evolved from a basic model to an improved and more reliable final solution, as required in the guideline.

3.1. Experiment 1 - Baseline CNN Model

In Implementation 1, a custom CNN was developed as the baseline model. The preprocessing stage included resizing all images to 224 × 224 pixels, normalizing pixel values to the range 0 to 1, converting the images into tensors, and preparing the dataset using TensorFlow's `image_dataset_from_directory()` function. Shuffling and prefetching were also applied to improve the data loading performance.

The baseline CNN contained three convolutional layers with 16, 32, and 64 filters. Each convolutional layer was followed by a max pooling layer. After feature extraction, the output was flattened and passed to a dense layer with 128 neurons. A dropout layer with a rate of 0.3 was used before the final softmax output layer. This model was selected because CNNs are suitable for image classification and provide a clear starting point for comparison with more advanced methods.

Table 2:Baseline Model Summary

Component	Details
Input size	224 × 224 × 3
Preprocessing	Rescaling by 1/255
Feature extraction	Conv2D (16), Conv2D (32), Conv2D (64)
Pooling	MaxPooling after each convolution block
Classifier	Flatten → Dense(128) → Dropout(0.3) → Softmax
Output classes	5

This model performed reasonably well and successfully learned the main visual features of the food classes. However, the training and validation curves showed that the model had limited generalization ability and began to show signs of overfitting in the later epochs.

```
Before normalization:
Min: 0.0, Max: 255.0

After normalization:
Min: 0.00, Max: 1.00
```

Figure 5: Normalization output

Model: "Food_Image_Classification"

Layer (type)	Output Shape	Param #
rescaling_1 (Rescaling)	(None, 224, 224, 3)	0
conv2d_3 (Conv2D)	(None, 222, 222, 16)	448
max_pooling2d_3 (MaxPooling2D)	(None, 111, 111, 16)	0
conv2d_4 (Conv2D)	(None, 109, 109, 32)	4,640
max_pooling2d_4 (MaxPooling2D)	(None, 54, 54, 32)	0
conv2d_5 (Conv2D)	(None, 52, 52, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 26, 26, 64)	0
flatten_1 (Flatten)	(None, 43264)	0
dense_2 (Dense)	(None, 128)	5,537,920
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 5)	645

Total params: 5,562,149 (21.22 MB)

Trainable params: 5,562,149 (21.22 MB)

Non-trainable params: 0 (0.00 B)

Figure 6:Baseline CNN model summary

3.2. Experiment 2 - Improved Model

During the early stage of Implementation 2, several attempts were made to improve the baseline CNN model. Different training configurations were tested, including increasing the number of epochs and adjusting some model parameters. However, these experiments did not produce significant improvements in performance. The accuracy remained relatively low compared to the desired level, and the training and validation graphs showed unstable learning behaviour. In particular, the validation accuracy did not increase consistently and the validation loss fluctuated, indicating that the model was not generalizing well.

Because of these limitations, a different approach was considered. Instead of relying only on a custom CNN, transfer learning was explored as a possible solution. Transfer learning allows the model to use knowledge learned from very large datasets and apply it to a new task. After reviewing several pretrained models commonly used for image classification, MobileNetV2 was selected because it is lightweight, efficient, and known to perform well for image recognition tasks. MobileNetV2 is also designed to provide strong feature extraction while keeping computational cost relatively low.

Therefore, the baseline CNN was replaced with a MobileNetV2 based transfer learning model in Implementation 2. This decision significantly improved the model's ability to extract meaningful features from food images and resulted in a large increase in classification accuracy compared to the original CNN model.

In Implementation 2, the baseline model was replaced with a transfer learning model based on MobileNetV2. This change was made because the baseline CNN had limited feature extraction ability, while pretrained networks such as MobileNetV2 can capture richer and more complex visual patterns from images.

The MobileNetV2 base model was loaded with ImageNet weights and configured with `include_top=False`. To reduce training cost and preserve useful pretrained knowledge, the earlier layers were frozen and only the later layers were fine tuned. On top of the base model, global average pooling, dropout layers, and a dense layer with 128 neurons were added, followed by a softmax output layer for the five food classes.

In addition to changing the architecture, several improvements were introduced,

- data augmentation
- class weighting
- dropout regularization
- early stopping
- learning rate reduction using `ReduceLROnPlateau`

These improvements were added to reduce overfitting, improve class balance during training, and increase generalization performance.

Table 3:Improved Model Summary

Component	Details
Base model	MobileNetV2 (ImageNet pretrained)
Input size	224 × 224 × 3
Data augmentation	RandomFlip, RandomRotation, RandomZoom, RandomContrast
Fine tuning	Early layers frozen, later layers trainable
Pooling	GlobalAveragePooling2D
Classifier	Dropout(0.4) → Dense(128) → Dropout(0.3) → Softmax
Output classes	5

This second experiment produced a clear improvement over the baseline model and became the final selected model for the project.

```
Found 2329 files belonging to 5 classes.
Found 498 files belonging to 5 classes.
Found 504 files belonging to 5 classes.

Classes: ['Chapathi', 'Dhal', 'Idli', 'Jelebi', 'Samosa']
Number of classes: 5
```

Figure 7:Classes and number of classes in Implementation 2

Figure 8:Sample training images for Implementation 2

```
Class Weights:
Chapathi: 0.8531
Dhal: 0.9080
Idli: 0.9526
Jelebi: 1.0586
Samosa: 1.3660
```

Figure 9:Class weights output

4. Training Process

The training process was different in the two implementations. The first implementation was trained as a basic CNN model for 10 epochs with a batch size of 32 using the Adam optimizer and sparse categorical crossentropy loss. The main purpose of this stage was to develop a correct baseline system and evaluate its behavior. The first implementation used validation data to monitor learning, but it did not include advanced optimization methods such as early stopping or learning rate scheduling. The Implementation 1 report confirms these baseline training details and shows the corresponding training and validation curves.

In the second implementation, the training process was improved to obtain better performance and better generalization. The improved model was trained for 12 epochs with a batch size of 32. The Adam optimizer was used with a learning rate of 0.0001. Sparse categorical crossentropy was again used as the loss function, and accuracy was used as the evaluation metric. During training, class weights were applied to reduce the effect of class imbalance. Early stopping was used to monitor validation loss and restore the best model weights when improvement stopped. In addition, `ReduceLROnPlateau` reduced the learning rate automatically when validation loss stopped improving.

Table 4:Training Configuration Comparison

Setting	Implementation 1	Implementation 2
Model	Custom CNN	MobileNetV2 Transfer Learning
Epochs	10	12
Batch size	32	32
Optimizer	Adam	Adam
Learning rate	Default Adam	0.0001
Loss function	Sparse Categorical Crossentropy	Sparse Categorical Crossentropy
Data augmentation	No	Yes
Class weights	No	Yes
Early stopping	No	Yes
ReduceLROnPlateau	No	Yes
Dropout	0.3	0.4 and 0.3

The training logs of Implementation 2 show that the improved model learned quickly and achieved high validation accuracy while keeping the validation loss low. The best weights were restored from epoch 10, which means the model did not need unnecessary extra training once it had already reached its best validation performance.

```
Epoch 1/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 121s 1s/step - accuracy: 0.7213 - loss: 0.7602 - val_accuracy: 0.8494 - val_loss: 0.4304 - learning_rate: 1.0000e-04
Epoch 2/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 88s 1s/step - accuracy: 0.9386 - loss: 0.1872 - val_accuracy: 0.8735 - val_loss: 0.3277 - learning_rate: 1.0000e-04
Epoch 3/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 88s 1s/step - accuracy: 0.9626 - loss: 0.1042 - val_accuracy: 0.9257 - val_loss: 0.1887 - learning_rate: 1.0000e-04
Epoch 4/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 88s 1s/step - accuracy: 0.9734 - loss: 0.0856 - val_accuracy: 0.9518 - val_loss: 0.1200 - learning_rate: 1.0000e-04
Epoch 5/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 143s 1s/step - accuracy: 0.9820 - loss: 0.0694 - val_accuracy: 0.9538 - val_loss: 0.1098 - learning_rate: 1.0000e-04
Epoch 6/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 89s 1s/step - accuracy: 0.9901 - loss: 0.0372 - val_accuracy: 0.9618 - val_loss: 0.0962 - learning_rate: 1.0000e-04
Epoch 7/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 89s 1s/step - accuracy: 0.9867 - loss: 0.0428 - val_accuracy: 0.9699 - val_loss: 0.0992 - learning_rate: 1.0000e-04
Epoch 8/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.9923 - loss: 0.0253
Epoch 8: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-05.
73/73 ━━━━━━━━━━━━━━━━━━━━ 88s 1s/step - accuracy: 0.9936 - loss: 0.0260 - val_accuracy: 0.9719 - val_loss: 0.0968 - learning_rate: 1.0000e-04
Epoch 9/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 89s 1s/step - accuracy: 0.9931 - loss: 0.0213 - val_accuracy: 0.9719 - val_loss: 0.0787 - learning_rate: 5.0000e-05
Epoch 10/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 89s 1s/step - accuracy: 0.9948 - loss: 0.0190 - val_accuracy: 0.9759 - val_loss: 0.0640 - learning_rate: 5.0000e-05
Epoch 11/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 89s 1s/step - accuracy: 0.9953 - loss: 0.0182 - val_accuracy: 0.9819 - val_loss: 0.0687 - learning_rate: 5.0000e-05
Epoch 12/12
73/73 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.9982 - loss: 0.0096
Epoch 12: ReduceLROnPlateau reducing learning rate to 2.499999936844688e-05.
73/73 ━━━━━━━━━━━━━━━━━━━━ 88s 1s/step - accuracy: 0.9966 - loss: 0.0142 - val_accuracy: 0.9819 - val_loss: 0.0680 - learning_rate: 5.0000e-05
Restoring model weights from the end of the best epoch: 10.
```

Figure 10:Training log of Implementation 2 with learning rate reduction and best epoch restoration

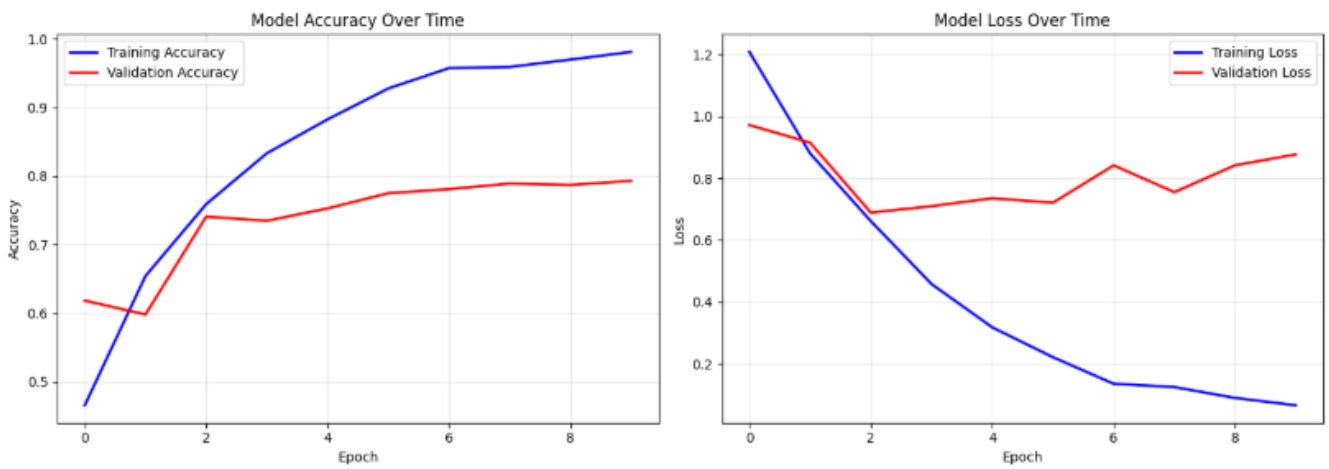

Figure 11:Accuracy and loss curves of Implementation 1

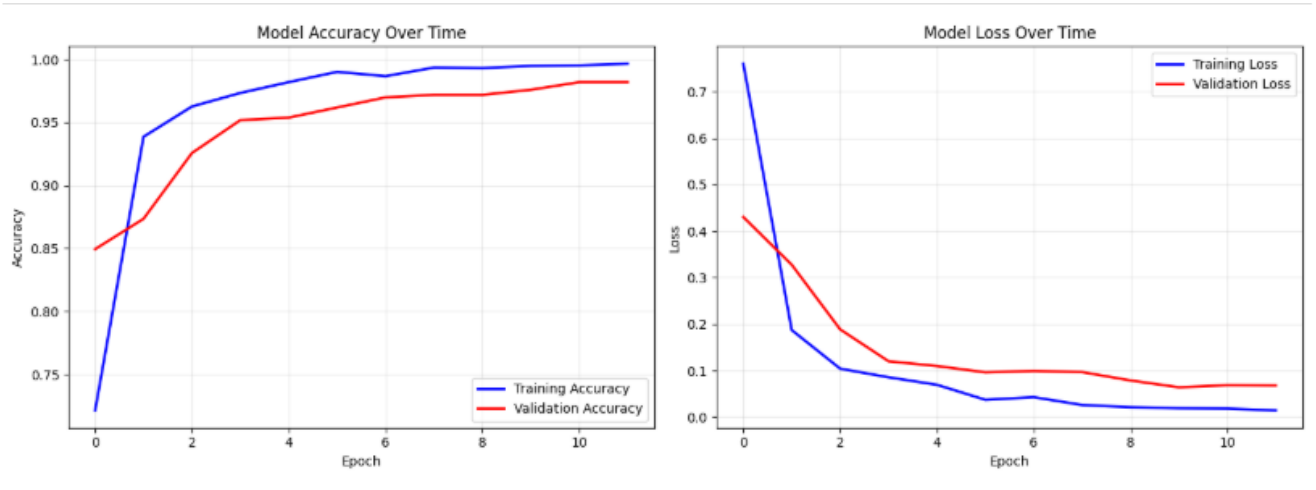

Figure 12:Accuracy and loss curves of Implementation 2

5. Experimental Results

This section presents the experimental results obtained from both implementations of the image classification model. The performance of each model is evaluated using training and validation curves, classification metrics, and confusion matrix analysis. These results help to compare the effectiveness of the baseline CNN model and the improved model, and clearly demonstrate how the performance improved after applying model optimization techniques.

5.1. Results of Implementation_1

The baseline CNN in Implementation 1 achieved a test accuracy of 81.75% with a test loss of 0.9891. The classification report showed that the model performed well on Chapathi and Idli, but weaker performance was observed for Samosa. The confusion matrix confirmed that most predictions were correct, but there were still several misclassifications between visually similar categories. The weighted average F1-score was approximately 0.814.

Table 5:Implementation 1 Test Results

Metric	Value
Test Accuracy	81.75%
Test Loss	0.9891
Weighted Precision	0.816
Weighted Recall	0.817
Weighted F1-score	0.814

5.2. Results of Implementation_2

The improved model achieved a test accuracy of 98.21% with a test loss of 0.0571. This is a very large improvement over the baseline model. The class wise precision, recall, and F1-scores were all very high. In particular, the Samosa class, which was weaker in Implementation 1, showed clear improvement. The confusion matrix also became much cleaner, with most predictions appearing along the diagonal. Only a very small number of images were misclassified.

Table 6:Implementation 2 Test Results

Metric	Value
Test Accuracy	98.21%
Test Loss	0.0571
Weighted Precision	0.983
Weighted Recall	0.982
Weighted F1-score	0.982

5.3. Class Wise Result Comparison

Class	Impl. 1 F1-score	Impl. 2 F1-score
Chapathi	0.883	0.987
Dhal	0.802	0.991
Idli	0.872	0.986
Jelebi	0.812	0.969
Samosa	0.641	0.973

The results show that all classes improved in Implementation 2. The biggest improvement was observed in the Samosa class, which moved from being the weakest class in the baseline model to one of the strong performing classes in the final model.

```
Test Accuracy: 81.75%
Test Loss: 0.9891
```

Figure 13:Test accuracy and loss output of Implementation 1

```
Classification Report:
==============================================================
              precision    recall  f1-score   support

    Chapathi      0.845     0.924     0.883       118
        Dhal      0.821     0.784     0.802       111
        Idli      0.876     0.868     0.872       106
      Jelebi      0.766     0.863     0.812        95
      Samosa      0.737     0.568     0.641        74

    accuracy                          0.817       504
   macro avg      0.809     0.801     0.802       504
weighted avg      0.816     0.817     0.814       504
```

Figure 14:Classification report of Implementation 1

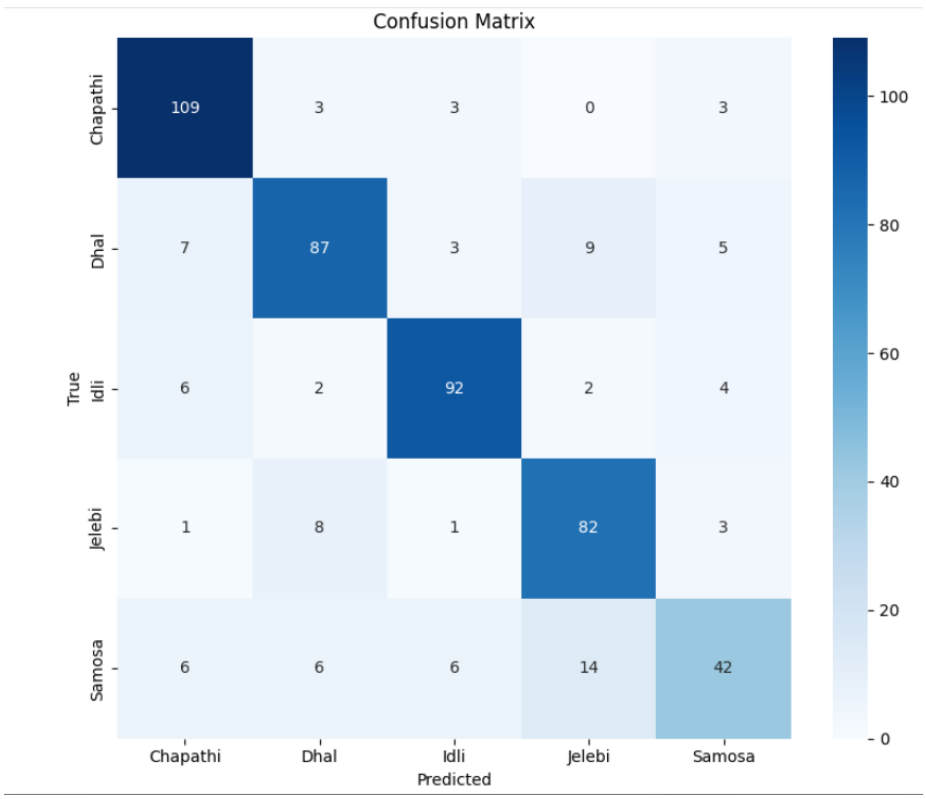

Figure 15:Confusion matrix of Implementation 1

Figure 16:Sample predictions of Implementation 1

```
Test Accuracy: 98.21%
Test Loss: 0.0571
```

Figure 17:Test accuracy and loss output of Implementation 2

```
Classification Report:
======================================================================
              precision    recall  f1-score   support

    Chapathi      0.975     1.000     0.987       118
        Dhal      1.000     0.982     0.991       111
        Idli      1.000     0.972     0.986       106
      Jelebi      0.949     0.989     0.969        95
      Samosa      0.986     0.959     0.973        74

    accuracy                          0.982       504
   macro avg      0.982     0.981     0.981       504
weighted avg      0.983     0.982     0.982       504
```

Figure 18:Classification report of Implementation 2

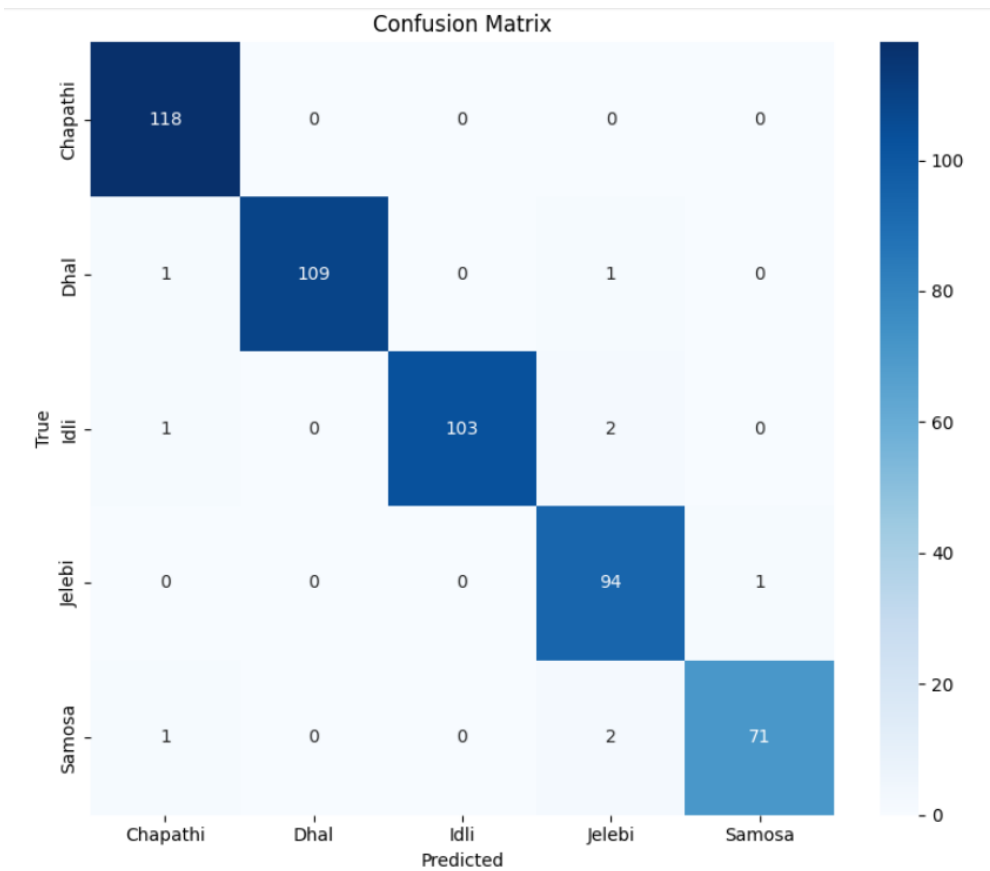

Figure 19:Confusion matrix of Implementation 2

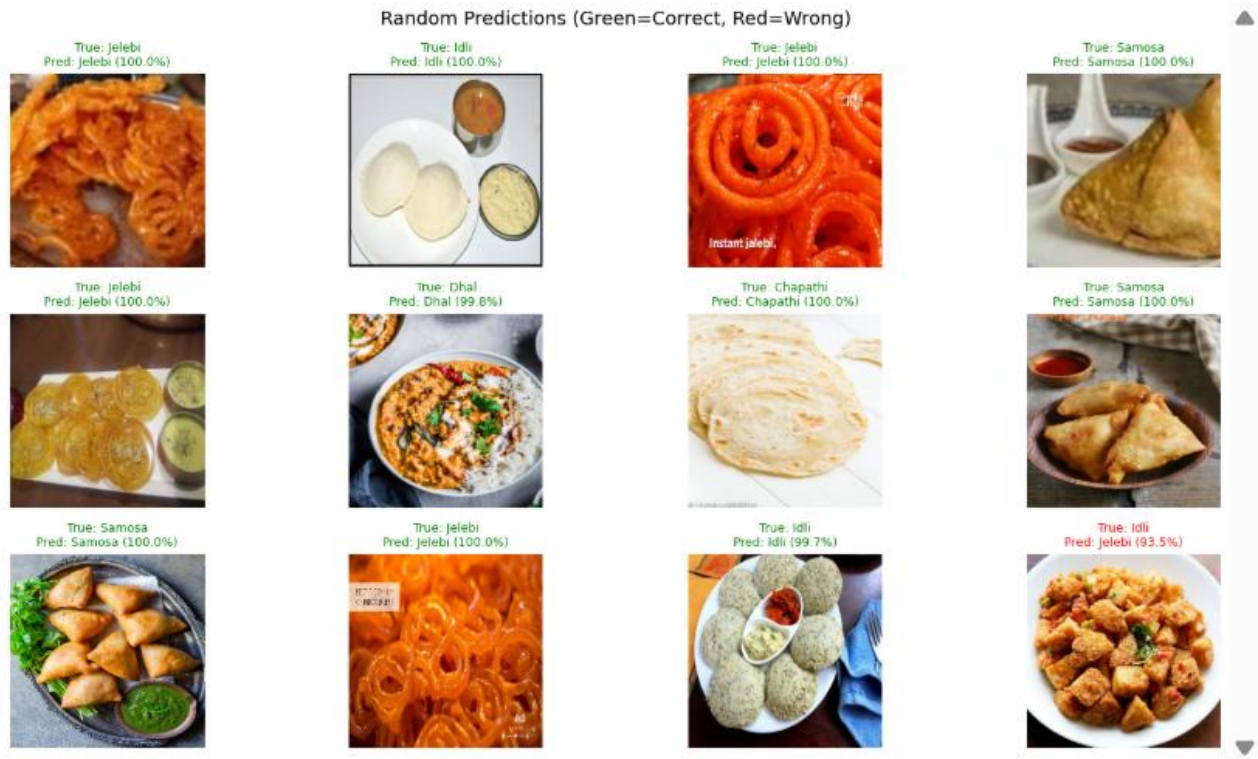

Figure 20:Sample predictions of Implementation 2

6. Overfitting Analysis

In Implementation 1, the training accuracy increased to a very high value while validation accuracy remained lower and stabilized around the upper 70% range. At the same time, the training loss continued to decrease, but the validation loss increased slightly during later epochs. This pattern indicates minor overfitting. The model was learning the training data well, but its ability to generalize to unseen data was limited.

The classification report and confusion matrix also support this observation. Although the model performed reasonably well overall, some classes showed much weaker recall and F1-scores, especially Samosa. This suggests that the model was not extracting strong enough features for all classes equally.

In Implementation 2, overfitting was reduced significantly. The training and validation curves remained much closer to each other. Validation accuracy improved strongly, and validation loss stayed low and stable. This indicates that the model generalized much better than the baseline CNN.

The reduction of overfitting was achieved through several methods,

- using MobileNetV2 pretrained features
- applying data augmentation
- adding dropout regularization
- using class weights
- stopping training early when validation loss stopped improving
- reducing the learning rate when improvement slowed down

These changes made the second model more stable and more effective on unseen test data.

7. Final Model and Results

The final selected model for this project was the MobileNetV2 based transfer learning model from Implementation 2. This model was chosen because it gave the best overall performance, reduced overfitting, improved weak classes, and produced much more reliable predictions on the test set.

The final model used data augmentation, class weighting, dropout, early stopping, and learning rate scheduling in addition to the pretrained MobileNetV2 architecture. These changes worked together to improve both performance and generalization. The final test accuracy of 98.21% and the very low test loss of 0.0571 show that the selected model is highly effective for the five food categories used in this project.

The model was saved as `model_v2.h5`, which allows it to be reused later without repeating the entire training process. This is important for future prediction, further testing, or integration into another system.

Table 7:Final Selected Model Details

Item	Final Model
Model name	Food_Image_Classifier
Saved file	model_v2.h5
Test Accuracy	98.21%
Test Loss	0.0571
Number of classes	5
Final choice reason	Best accuracy, low loss, better generalization

Model: "Food_Image_Classifier"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
data_augmentation (Sequential)	(None, 224, 224, 3)	0
true_divide (TrueDivide)	(None, 224, 224, 3)	0
subtract (Subtract)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 128)	163,968
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 5)	645

Total params: 2,422,597 (9.24 MB)

Trainable params: 2,026,053 (7.73 MB)

Non-trainable params: 396,544 (1.51 MB)

Figure 21:Improved model summary

```
Model successfully saved!
Save location: C:\Users\LENOVO\Desktop\IIS_Project\model_v2.h5
```

Figure 22:Final model saving confirmation

8. Discussion

This project clearly showed how an image classification system can be improved step by step. The baseline CNN in Implementation 1 was useful because it established the complete workflow and helped identify the main weaknesses of the system. It showed that the model could classify most images correctly, but it also revealed the limitations of a simple architecture.

The second implementation improved the system considerably. The use of MobileNetV2 was the most important change because it provided much stronger feature extraction than the custom CNN. Data augmentation helped the model become more robust to image variations. Class weights improved fairness across classes, especially for the smaller Samosa class. Dropout, early stopping, and learning rate reduction helped stabilize training and reduce overfitting.

What worked well in this project was the combination of transfer learning and training optimization methods. What did not work well in the baseline stage was the limited ability of the simple CNN to capture difficult visual differences across all classes. Some challenges faced during the project were dataset imbalance, visual similarity among food images, and variations in lighting, angle, and background. A major lesson learned from this work is that model architecture alone is not enough; preprocessing, regularization, evaluation, and training strategies are also very important for achieving a reliable result.

Table 8:Justification of Improvements

Problem	Solution	Justification	Impact
Weak feature extraction from small dataset	MobileNetV2 transfer learning	Pretrained on 14M images provides rich features without learning from scratch	Accuracy ↑ from 81.75% to 98.21%
Samosa underperformed (0.641 F1)	Class weights (Samosa: 1.66)	Forces model to treat minority classes equally	Samosa F1 ↑ to 0.973 (+51.8%)
Overfitting (train-val gap)	Augmentation + Dropout + Early Stopping	Creates variations, prevents neuron co-dependence, saves best weights	Test loss ↓ from 0.9891 to 0.0571
Training plateaus	Lower LR + ReduceLROnPlateau	Gentle fine tuning; adapts when stuck	Stable convergence
Poor generalization	Combined all techniques	Each fix supported others	All classes >0.96 F1

9. Conclusion and Future Work

This project successfully developed an intelligent food image classification system in two stages. The first stage created a baseline CNN model that achieved a test accuracy of 81.75%. This model established the basic workflow correctly but showed minor overfitting and weaker performance in some classes. The second stage introduced a MobileNetV2 based transfer learning model together with several optimization techniques. As a result, the final model achieved 98.21% test accuracy and much stronger class wise performance.

Overall, the project met its objective of building and improving an image classification model while also documenting the full learning process from baseline to optimized solution. This matches the reporting requirement that the final report must show how the model evolved and why the final version works better.

However, the current model still has limitations. It is trained only on five food classes, and its performance may reduce when tested on real world images with very different backgrounds, camera quality, or presentation styles. In future work, the dataset can be expanded, more food categories can be added, and other pretrained models such as EfficientNet or ResNet can be tested. The model could also be deployed in a web or mobile application for practical use.

10. GitHub Link

https://github.com/SKeerthy/Food_Image_Classififcation_System.git